4.1 Introduction (Input / Output – I/O)

What is I/O?

Input/Output (I/O) means **data transfer between a microcomputer and external devices**. A user communicates with a microcomputer using **I/O devices**.

Examples of I/O devices (Peripherals)

These devices connect the microcomputer to the outside world:

- Keyboard
- Monitor (screen)
- Printer
- Hard disk

These devices are called **peripherals**.

Why do we need an Interface?

I/O devices and microcomputers are **different**. Peripherals are usually **slower** than the microcomputer, and data size (word length) may be **different**. So, **interface hardware** is required between the microcomputer and I/O devices.

Functions of Interface

- Makes I/O devices **compatible** with the microcomputer
- Handles all data transfer using an **I/O bus**

I/O Bus Signals

An **I/O bus** carries three types of signals:

- **Address** – selects the device
- **Data** – actual information
- **Command** – tells what action to perform

I/O Instruction

A microprocessor uses the **I/O bus** when it executes an **I/O instruction**. An I/O instruction has **three fields**.

Fields of I/O Instruction

- Op-code
- Device address
- Command

Working

- Control unit identifies it as an I/O instruction
- Device address and command are placed on the I/O bus
- Correct interface is selected
- Data is either:
    - Read from input device, or
    - Sent to output device

Types of I/O Devices

There are **two types**:

1. Physical I/O

- Used when **no operating system** is present
- User works **directly with hardware**
- Requires detailed I/O design

2. Virtual I/O

- Used when an **operating system (OS)** is present
- User does **not worry about hardware details**
- OS handles everything

Methods of Data Transfer

There are **three ways**:

1. Programmed I/O

- Microprocessor controls **all data transfer**

- Uses a program stored in memory
- **Simple but slow**

2. Interrupt I/O

- External device sends an **interrupt signal**
- Microprocessor:
  - Stops current program
  - Executes **Interrupt Service Routine (ISR)**
  - Returns to main program

3. Direct Memory Access (DMA)

- Data transfers directly between:
  - Memory and I/O device
- **Microprocessor is not involved**
- Uses a **DMA controller**
- **Fastest method**

Role of Operating System in I/O

The operating system acts as an **interface between user and hardware** and creates **virtual (logical) I/O devices**.

Example: Spooling

Output is sent to disk first (virtual printer) and later printed using an actual printer. This improves **system speed**. The user does not know which physical printer is used. This gives **flexibility and better performance**.

4.2 Simple I/O Devices

Simple Input Device – DIP Switch

A **DIP switch** can be used as an input device. When the switch is **open**, the input is **HIGH**, and when the switch is **closed**, the input is **LOW**. It is used for **simple lab experiments**.

Protection

- A **1 kΩ resistor** is used for protection from static electricity and MOS gate safety

Simple Output Device – LED

An **LED (Light Emitting Diode)** is commonly used. It requires **low voltage** and **low current**.

LED Requirements

- Red → 10 mA, 1.7 V
- Yellow → 10 mA, 2.2 V
- Green → 20 mA, 2.4 V

Working of LED

The LED glows when the cathode is **negative** to the anode. The microcomputer can apply **+5 V through a resistor** or use a **buffer/inverter**.

Why Buffer is Needed?

The microcomputer output current is **very small**, while the LED needs **more current**, so a **buffer (inverter)** is used.

Resistor Calculation (Exam Important!)

Given supply voltage = 5 V, LED voltage = 1.7 V, and LED current = 10 mA.
R = {5 - 1.7}/{10mA} = 330 \OmegaThe required resistor is **330 Ω**.

Seven-Segment Display

A seven-segment display is used to display **digits 0 to 9**. It contains **7 LEDs** labeled a–g.

Example

To display **0**, segments **a–f are ON** and segment **g is OFF**.

Types of Seven-Segment Displays

1. Common Cathode

- HIGH → segment ON
- LOW → segment OFF

2. Common Anode

- LOW → segment ON
- HIGH → segment OFF

Interfacing Seven-Segment Display

Seven-segment displays can be interfaced using hardware chips such as **7447 and 7448**, or by using a **software look-up table** in memory. An assembly program reads data and outputs it to the display.

4.3 Programmed I/O—What is Programmed I/O?

In **programmed I/O**, the **microcomputer controls all data transfer** using a program. The microcomputer communicates with external devices through **I/O ports**. These ports contain **registers**.

I/O Ports

I/O ports are special registers used for input (reading data) and output (sending data).

Types of I/O Ports

There are **two types**:

- **Bit-configurable I/O ports**
- **Parallel I/O ports**

1. Bit-configurable I/O ports

Each **bit** can be set **individually** as input or output.

2. Parallel I/O ports

All bits together work as all inputs or all outputs.

Data-Direction (Command) Register

The data-direction register is used to decide whether port bits are **input or output**. It is an **output register**. The actual data is stored in the **I/O port**.

Rule

Writing **0 or 1** in a bit of the data-direction register defines the corresponding port bit as **input or output**.

Example (Very Important for Exam)

Given

- Data-direction register = **34H**
- Binary of 34H = **0011 0100**

Bit No	7	6	5	4	3	2	1	0
Value	0	0	1	1	0	1	0	0

Interpretation

- **Input bits** → 0, 1, 3, 6, 7
- **Output bits** → 2, 4, 5

Usage

- Output devices (LEDs) → bits **2, 4, 5**
- Input devices (switches) → bits **0, 1, 3, 6, 7**

Important Note (Exam Point)

While reading input, the microcomputer reads the **full byte**. Output bits contain **irrelevant values**. These are treated as **"don't care"**. Outputs to input-configured bits are **disabled**.

Parallel I/O Ports

Only **one data-direction register** controls all ports. One bit in the direction register configures **all bits of one port** as input or output.

Handshake Ports

Handshake ports are special I/O ports. Data transfer uses **control signals**. This ensures proper synchronization between the microcomputer and the external device.

Addressing of I/O Ports

I/O ports can be addressed in **two ways**:

- **Standard I/O (Port-Mapped or Isolated I/O)**
- **Memory-Mapped I/O**

1. Standard I/O (Port-Mapped or Isolated I/O)

Features

- Uses a special control pin (**M/IO or M/E**)
- Pin value:
    - **HIGH → Memory operation**
    - **LOW → I/O operation**

Instructions

- **IN** and **OUT** → I/O
- **MOVE** → Memory

Addressing

- Usually **8-bit address**
- Can access **256 I/O ports**
- 32-bit processors may use **16- or 32-bit ports**

Example

```
IN  AL, PORTA
OUT PORTA, AL
```
IN reads data from PORTA and OUT sends data to PORTA.

2. Memory-Mapped I/O

Features

- No special I/O pin used
- I/O ports are treated as **memory locations**
- Uses **memory instructions** like MOVE

Key Idea

Some memory addresses are **assigned to I/O devices**. These addresses may not exist physically.

Used By

- **Motorola 68000 / 68020** → only memory-mapped I/O
- **Intel processors** → both methods

Example

```
MOV mem, reg
MOV reg, mem
```
This reads from or writes to an I/O port mapped as memory.

Quick Exam Summary

- Programmed I/O uses **software control**

- I/O ports communicate with external devices
- Data-direction register defines input/output
- Two addressing methods:
  - Standard I/O
  - Memory-mapped I/O
- IN/OUT → standard I/O
- MOV → memory-mapped I/O

4.4 Unconditional and Conditional Programmed I/O

Programmed I/O – Two Types

Programmed I/O can be used in **two ways**:

- **Unconditional I/O**
- **Conditional I/O**

1. Unconditional Programmed I/O

Meaning

The microprocessor sends data **without checking** whether the external device is ready. The external device **must always be ready**.

Key Point

There is no checking of status signals. This method is simple and fast.

Example (Exam Friendly)

Sending a **7-bit code** to a **seven-segment display**. The display always accepts data, so no checking is needed. This method is **used when the device is always ready**.

2. Conditional Programmed I/O

Meaning

The microprocessor sends or receives data **only when the device is ready**. Readiness is checked using **status signals**.

How is readiness checked?

- By **handshaking**

Handshaking is the exchange of **control signals** between the microprocessor and the external device.

Process

1. Microprocessor checks the **status bit** of the device.
2. If device is ready, data transfer occurs.
3. If not ready, the microprocessor **waits in a loop**.

This idea is shown using a flowchart (Fig. 4.6).

Example: A/D Converter (Very Important)

What does an A/D Converter do?

An A/D converter converts an **analog voltage (Vx)** into an **8-bit digital output**.

Important Signals

- **Start** → begins conversion
- **Conversion Complete**
  - LOW → conversion in progress
  - HIGH → conversion finished
- **Output Enable**
  - LOW → data available at output
  - HIGH → output disabled (tri-state)

Conditional I/O with A/D Converter

Working Steps

1. Microcomputer sends a **start pulse** to A/D converter.
2. Converter starts conversion.
3. Microcomputer **keeps checking** conversion-complete signal.
4. When conversion-complete becomes HIGH:
   - Microcomputer sends LOW to output enable.

- Digital data is read from the converter.

If conversion not complete

The microcomputer waits in a **loop** checking the signal. This is **conditional programmed I/O** because data transfer depends on device status.

4.5 Interrupt I/O

Problem with Conditional Programmed I/O

The microcomputer **wastes time waiting** in a loop. For **slow devices**, this reduces system performance. The solution is **Interrupt I/O**.

What is Interrupt I/O?

Interrupt I/O is **device-initiated I/O**. The external device tells the microcomputer, *"I am ready now!"* using an **Interrupt (INT) pin**.

Basic Working of Interrupt I/O

1. External device activates the **INT pin**.
2. Microcomputer finishes the current instruction and saves:
   - Program Counter (PC)
   - Status Register (SR)
     in the **stack**.
3. Microcomputer jumps to a special program called **Interrupt Service Routine (ISR)**.
4. ISR performs required data transfer.
5. ISR ends with **Return from Interrupt (RTE)**.
6. Microcomputer restores PC and SR and continues the main program.

This process is shown in Fig. 4.9.

Interrupt I/O with A/D Converter (Example)

Initial Program (Main Program)

The main program configures Port A bits 0 and 7 as outputs and Port B as input. It sends a start pulse to the A/D converter, sets output enable HIGH (disable output), and clears a register.

Important Note

- `.B` → byte operation
- `.W` → word operation
- `$` → hexadecimal
- `#` → immediate value

When Conversion Completes

When conversion complete becomes HIGH, it activates the **INT pin**. The microcomputer completes the current instruction and jumps to the ISR.

Interrupt Service Routine (ISR)

Written by the user

```
ORG $4000
MOVE.B #$00, PORTA   ; Enable A/D output
MOVE.B PORTB, D1     ; Read digital data
RTE                  ; Return from interrupt
```

Function of ISR

The ISR enables the A/D output, reads the digital value, and returns to the main program.

After RTE Instruction

After the RTE instruction, PC and SR are restored from the stack. The microcomputer continues the main program **from where it stopped**.

Exam Comparison (Very Important)

Feature	Conditional Programmed I/O	Interrupt I/O
Device checking	Polling (waiting loop)	Device interrupts CPU
CPU time	Wasted	Efficient
Speed	Slow	Faster
Suitable for	Fast devices	Slow devices

Short Exam Summary

- Unconditional I/O: no checking
- Conditional I/O: checks device status
- Interrupt I/O: device signals CPU

- ISR handles data transfer
- RTE returns to main program

4.5.1 Interrupt Types

There are **three main types of interrupts**:

- **External Interrupts**
- **Internal Interrupts (Traps)**
- **Software Interrupts**

1. External Interrupts

External interrupts are generated by **external devices** such as an A/D converter or a keyboard. These interrupts are sent to the microprocessor through **interrupt pins**.

Types of External Interrupts

- **Maskable Interrupt**
- **Non-Maskable Interrupt (NMI)**

(a) Maskable Interrupt

A maskable interrupt can be **enabled or disabled** by software. It is controlled by specific **instructions**.

Example (Intel Pentium):

- `CLI` → disables interrupt
- `STI` → enables interrupt

These instructions change the **interrupt flag** in the status register.

(b) Non-Maskable Interrupt (NMI)

A non-maskable interrupt **cannot be disabled**. It has **higher priority** than a maskable interrupt and is always serviced first.

Important Use

NMI is used during **power failure** conditions. It allows the system to save important data before shutdown. The data is stored in **non-volatile memory**, such as battery-backed RAM.

Exam Line:
If maskable and non-maskable interrupts occur together, **NMI is serviced first**.

Handshake Interrupt (Pentium)

Handshake interrupt uses **two pins**:

- `INTR` → interrupt request
- `INTA` → interrupt acknowledge

The external device sends an **8-bit number** to the CPU. The CPU uses this number to find the **interrupt service routine address**.

2. Internal Interrupts (Traps)

Internal interrupts are generated **inside the microprocessor**. They are caused by errors such as divide by zero, overflow, or illegal instruction. These interrupts are handled in the same way as external interrupts. The user must write a **service routine** to handle the error.

3. Software Interrupts

Software interrupts are generated by executing a **special instruction**. They are used to **call the operating system** and are also known as **system calls**.

Advantages

- Shorter than subroutine calls
- No need to know operating system memory address
- Allows switching from **user mode to supervisor mode**

In some processors, this is the **only way** to access the operating system.

4.5.2 Interrupt Address Vector

The interrupt address vector is the **starting address of the interrupt service routine (ISR)**.

Two Methods

- **Fixed address**
- **Indirect method**

In the fixed address method, the manufacturer decides the ISR address. In the indirect method, the ISR address is stored in a **fixed memory location**, and the CPU reads it when an interrupt occurs. The method used depends on the processor.

4.5.3 Saving Microprocessor Registers

When an interrupt occurs, the microprocessor saves the **Program Counter (PC)** and the **Status Register (SR)** in the **stack**.

This is important so that the program can return to the **exact place** where it was interrupted and continue execution normally.

Exam Note

The user must know which registers are saved and which **return instruction** is used, such as `RTE`.

4.5.4 Interrupt Priorities

When many devices share **one interrupt pin**, the CPU must know **which device interrupted first**.

Two Ways to Handle Multiple Interrupts

- **Polled Interrupts**
- **Daisy Chain Interrupts**

1. Polled Interrupts

Polled interrupts are handled by **software** and are relatively **slow**.

Working

The CPU jumps to a **general service routine** and checks each device one by one. The order of checking determines the **priority**. When the interrupt source is found, the CPU jumps to that device's ISR.

Disadvantage

This method is time-consuming and not suitable when many devices are present.

Exam Line:
Priority is determined by the **polling order**.

Example: Polled Interrupt with A/D Converters

Devices are connected using an **OR gate**. The CPU checks device 2 first, which has higher priority, and then device 1. AND-gate outputs are used to detect the interrupt source.

2. Daisy Chain Interrupts

Daisy chain interrupts are handled by **hardware** and are **faster** than polling.

Working

Devices are connected in **series**. The CPU sends an **INTA** signal. The highest-priority device receives INTA first. If it is not requesting an interrupt, it passes INTA to the next device. The interrupting device accepts INTA and sends the interrupt address vector.

Daisy Chain Priority Rule

- **Nearest device to CPU → highest priority**
- **Last device → lowest priority**

Advantages

- Fast response
- No polling loop

Disadvantage

- Hardware complexity
- Failure of one device may affect others

Polled vs Daisy Chain (Very Important Table)

Feature	Polled Interrupt	Daisy Chain Interrupt
Method	Software	Hardware
Speed	Slow	Fast
Priority	Fixed by program	Fixed by position
Hardware	Simple	More complex
Suitable for	Few devices	Many devices

Final Exam Summary

- Three interrupt types: External, Internal, Software
- NMI has highest priority
- Interrupt vector gives ISR address
- PC and SR are saved on stack
- Multiple interrupts handled by:
    - Polling
    - Daisy chain

4.6 Direct Memory Access (DMA)

What is DMA?

Direct Memory Access (DMA) is a method of data transfer between memory and an I/O device without using the microprocessor. It is used when large amounts of data must be transferred. An example of DMA is data transfer between hard disk and RAM. DMA makes data transfer fast.

Why DMA is Needed

Programmed I/O and Interrupt I/O use the microprocessor and are slow for large data blocks. DMA transfers data directly, saves CPU time, and improves system performance.

DMA Controller

DMA uses a special chip called a DMA controller. It performs data transfer in hardware. The DMA controller has an address register, a counter register, and a status register, and it can transfer data very fast.

Main Functions of DMA Controller (Step-by-Step Operation)

First, the I/O device requests DMA using the DMA request line. Then the DMA controller sends a HOLD signal to the microprocessor. The microprocessor replies with HLDA (Hold Acknowledge). After that, the microprocessor releases the bus. The DMA controller takes control of the address, data, and control buses and sends DMA ACK to the I/O device. Data transfer occurs directly between memory and the I/O device. After completion, the DMA controller sends an INT signal to the CPU and releases the bus, and the CPU resumes normal operation.

Types of DMA

There are three types of DMA:

- **Block Transfer DMA**
- **Cycle Stealing DMA**
- **Interleaved DMA**

Block Transfer DMA

In block transfer DMA, the DMA controller takes full control of the bus and transfers the entire block of data. During this time, the microprocessor cannot access the bus and can perform internal operations only. This is the fastest DMA, it is most commonly used, and it is suitable for large data blocks.

Cycle Stealing DMA

In cycle stealing DMA, data is transferred one word at a time. The DMA controller steals one clock cycle at a time. It uses an INHIBIT signal where HIGH means the CPU runs normally and LOW means the CPU is stopped for one cycle. It is called cycle stealing because DMA steals a clock cycle and the CPU does not know it happened. In this method, the CPU is slowed slightly and data transfer takes more time.

Interleaved DMA

In interleaved DMA, the DMA uses the bus only when the CPU is not using it. For example, while the CPU is incrementing the program counter or while the CPU is doing ALU operations. In this type, there is no stopping of the CPU, data transfer is spread over time, and the logic is more complex.

Block Transfer DMA (Important for Exam)

The operation of block transfer DMA starts when the I/O device requests DMA. The DMA controller sends HOLD and the CPU sends HLDA. The DMA controller then controls the bus and data is transferred between RAM and the I/O device. After completion, the DMA controller sends an interrupt to the CPU and the CPU regains bus control.

Registers in DMA Controller

The registers in a DMA controller are:

- **Address Register**
  Holds starting memory address.
- **Terminal Count Register**
  Holds number of words to transfer.
- **Status Register**
  Shows DMA completion and errors.

These registers are initialized by the microprocessor.

Advantages of DMA

- Very fast data transfer
- CPU free for other work
- Best for large data blocks

Disadvantages of DMA

- Extra hardware needed
- CPU loses bus control temporarily

4.7 Summary of I/O (Figure 4.15 Explained)

I/O Techniques in a Microcomputer

There are three I/O techniques in a microcomputer:

- **Programmed I/O**

- **Interrupt I/O**
- **Direct Memory Access (DMA)**

Programmed I/O

In programmed I/O, the CPU controls everything. There are two types of programmed I/O:

- Standard (Port / Isolated I/O)
- Memory-Mapped I/O

Interrupt I/O

In interrupt I/O, the device signals the CPU. The types of interrupts are:

- External
    - Maskable
    - Non-maskable
- Internal (Traps)
- Software interrupts

Direct Memory Access (DMA)

In DMA, there is no CPU involvement. The types of DMA are:

- Block Transfer
- Cycle Stealing
- Interleaved

One-Line Exam Summary

DMA transfers data directly between memory and I/O device without CPU involvement. Block transfer DMA is fastest. DMA controller manages address, count, and data transfer. DMA improves system throughput.